

COMS W4771: Machine Learning

Lecture 24: Transformers

Tony Dear, Ph.D.

Department of Computer Science

School of Engineering and Applied Sciences

Topics

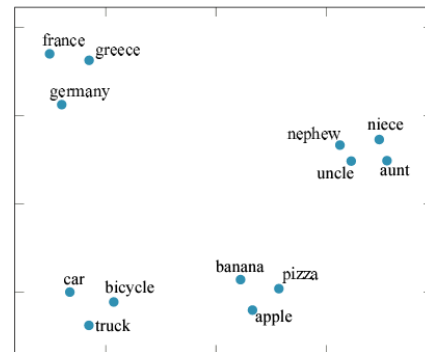
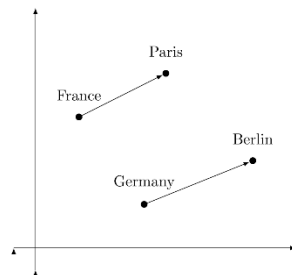
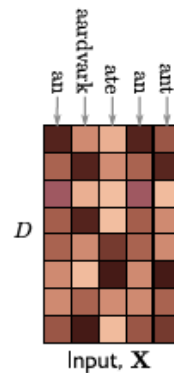
- Language modeling
- Self-attention
- Transformers
- Language and vision models

Tokenization

- Natural language and text are messy, high-dimensional, and variable-length
- **Tokenization** breaks words into pieces corresponding to some vocabulary
- Token granularity can range from individual letters up to full words
- Most tokenizers assign tokens to “sub-words” as a compromise, including common words as well as fragments from parts of speech, suffixes, etc.
 - Ex: “unhappiness”: [“un”, “happi”, “ness”], “tokenization”: [“token”, “iz”, “ation”]
- Punctuation, symbols, capital letters can all be tokenized separately
- Tokenization algorithms include *byte pair encoding* (BPE), *WordPiece*, etc.

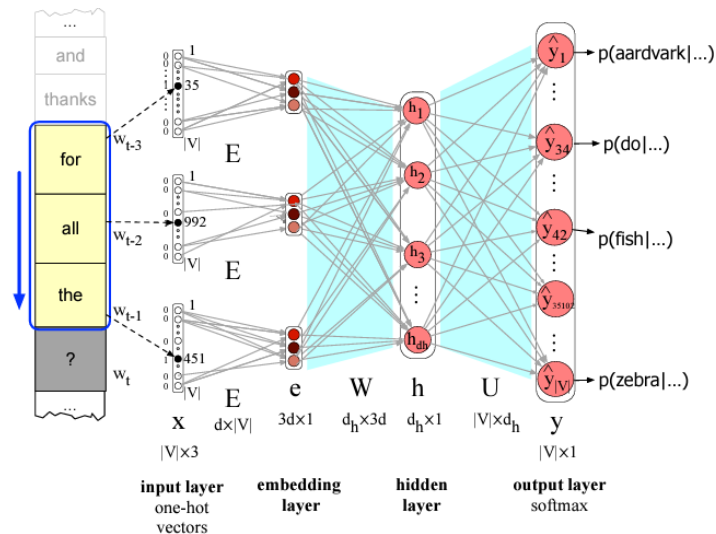
Embeddings

- Given N input tokens, we can represent them in an input **embedding** matrix $\mathbf{X} \in \mathbb{R}^{D \times N}$, where D is the dimension of the embedding
- Numerical vector representations of words and tokens
- Useful embeddings tend to exhibit word similarities in terms of vector similarities or closeness
- Vector *directions* may also reveal semantic relationships
- Embeddings are often trained alongside the actual machine learning models



Language Modeling

- **Language modeling** is the task of *predicting* upcoming words from prior words
- A **neural language model** is a neural network that takes in some number of words and outputs a probability distribution over possible next words
- As with image data, a fully connected network is not ideal for seeing *contextual* information
 - Ex: “The **animal** didn’t cross the street because **it** was too tired”
- The word “it” is ambiguous in isolation, but we know it aligns with “animal”



Queries, Keys, and Values

- Consider linear transformations of all tokens: *query* $\mathbf{q}(\mathbf{x})$, *key* $\mathbf{k}(\mathbf{x})$, *value* $\mathbf{v}(\mathbf{x})$
- The query $\mathbf{q}(\mathbf{x})$ is used to *compare* $\mathbf{x}$ with other tokens (e.g., as a search term)
- The key $\mathbf{k}(\mathbf{x})$ can be used to *answer* queries posed by other tokens

- Ideally, $\mathbf{q}(\text{"it"})$ should be well aligned with $\mathbf{k}(\text{"animal"})$
- “it” pulls information from “animal”, though not necessarily true in reverse

- Consider transforming $\mathbf{x}$ by “pulling in” context from other tokens $\mathbf{x}'$
- Context of each token is represented by the *values* $\mathbf{v}(\mathbf{x}')$, and the amount to “pull in” is given by similarity between $\mathbf{q}(\mathbf{x})$ and $\mathbf{k}(\mathbf{x}')$

Self-Attention

- Parameters: Weights $\mathbf{\Omega}_q, \mathbf{\Omega}_k, \mathbf{\Omega}_v \in \mathbb{R}^{D \times D}$, biases $\mathbf{\beta}_q, \mathbf{\beta}_k, \mathbf{\beta}_v \in \mathbb{R}^D$
- Given input matrix $\mathbf{X}$, compute query and key matrices $\mathbf{Q}(\mathbf{X}) = \mathbf{\beta}_q \mathbf{1}^\top + \mathbf{\Omega}_q \mathbf{X}$ and $\mathbf{K}(\mathbf{X}) = \mathbf{\beta}_k \mathbf{1}^\top + \mathbf{\Omega}_k \mathbf{X}$, followed by $\mathbf{S} = \mathbf{K}^\top \mathbf{Q} \in \mathbb{R}^{N \times N}$
- The i th column of $\mathbf{S}$ contains the dot products between $\mathbf{q}(\mathbf{x}_i)$ and keys $\mathbf{k}(\mathbf{x}_j)$
- Convert each column into a *probability distribution* using softmax:

$$\mathbf{A}_i = \text{Softmax} \left((\mathbf{K}^\top \mathbf{Q})_i / \sqrt{D} \right)$$

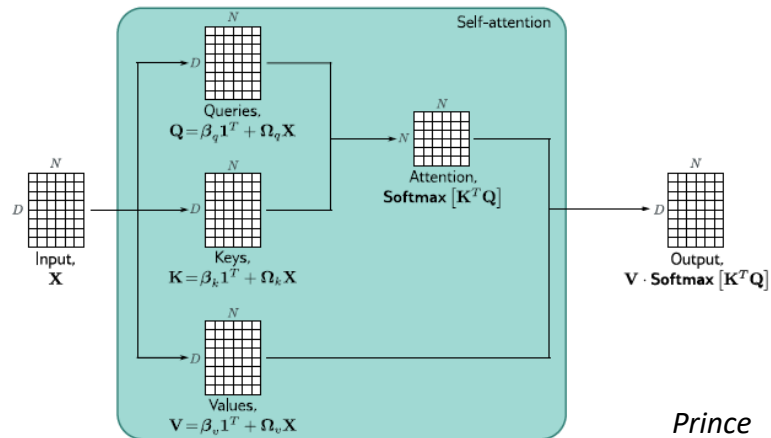
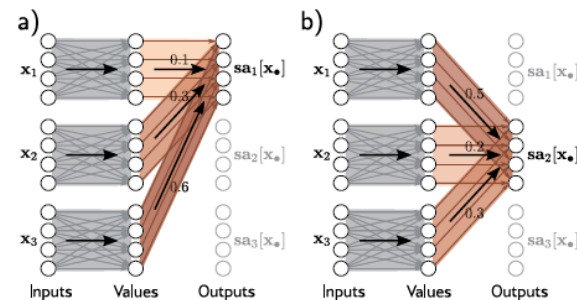
- The **self-attention weights** $\mathbf{A}$ contains the *relative* (asymmetric) similarities between each pair of tokens

Self-Attention

- The **self-attention outputs** are computed by using **A** to weight the value matrix $V(X) = \beta_v \mathbf{1}^T + \Omega_v X$:

$$\mathbf{SA}(X) = V(X)A \in \mathbb{R}^{D \times N}$$

- Think of *routing* the values $V(X)$ to outputs using proportions given by attention weights
- Nonlinear computation due to softmax
- This produces *contextualized* token outputs
- Each token has incorporated and “blended in” relevant information from the sequence



Positional Encodings

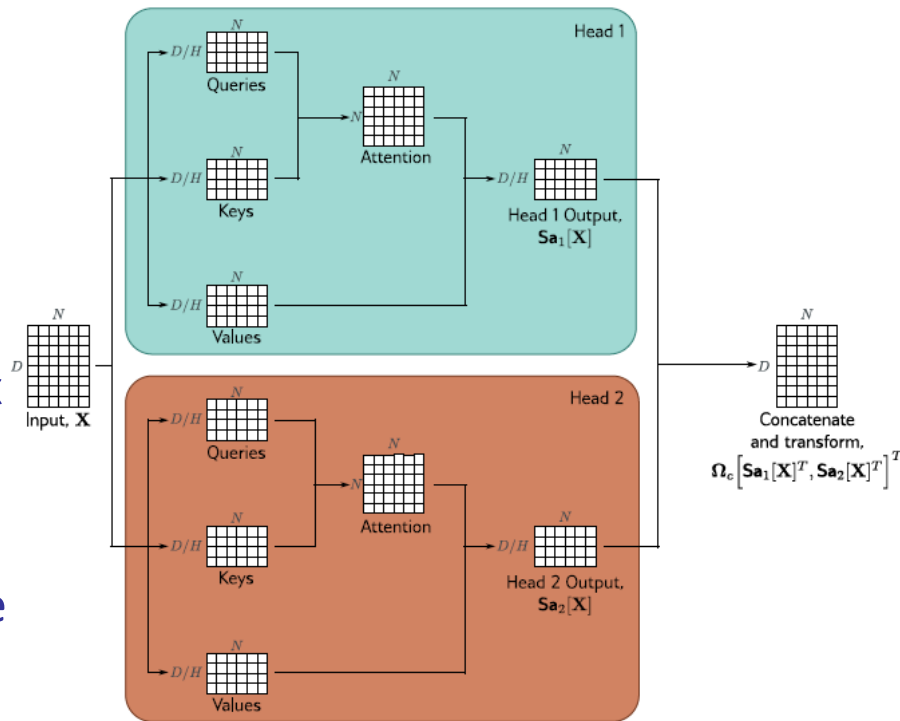
- The ordering of words and tokens is also important in many languages
- We should include sequence position in token representations
- A simple way is to add *positional encodings*: $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{\Pi}$, where the values of $\mathbf{\Pi}$ depend on the position of token in the sequence
- $\mathbf{\Pi}$ can be defined using fixed encoding functions, e.g. sinusoids
- Alternatively, positional encodings can also be learned alongside embeddings
- Position information can also be included in the attention calculation directly

Multi-head Self-Attention

- With language, there are often *multiple* kinds of relationships to look for: grammar, syntax, references, local context, long-range dependencies, etc.
- A single attention matrix is often insufficient for revealing everything
- **Multi-head self-attention:** Compute multiple attention outputs $\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h$, each using a different set of weights and biases
- Each head can specialize in a different kind of attention (context)
- Analogy to multiple filters or channel outputs in convolutional networks
- In both cases, we are learning multiple *representations* of the same input

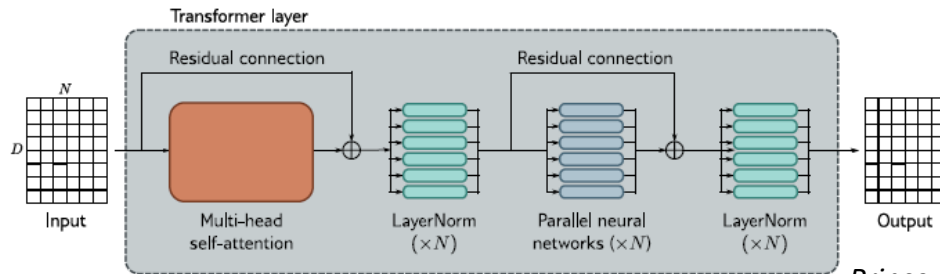
Multi-head Self-Attention

- With H heads, we typically set the size of each to be D/H
- The outputs are $D/H \times N$ matrices, each computed in parallel
- Stack everything into one $D \times N$ matrix
- Typically apply a final linear transform Ω_c to allow for *interaction* between the different head outputs



Transformer Layers

- A **transformer layer** consists of a multi-head self-attention unit followed by a fully connected, feed-forward network applied to each output token
- Both units contain **residual connections**—add inputs directly to the outputs
- So the layer simply *shifts* the input instead of completely changing it
- Also helps to avoid vanishing gradients and stabilize training
- The **LayerNorms** normalize each token according to learned scale and shift parameters



Prince

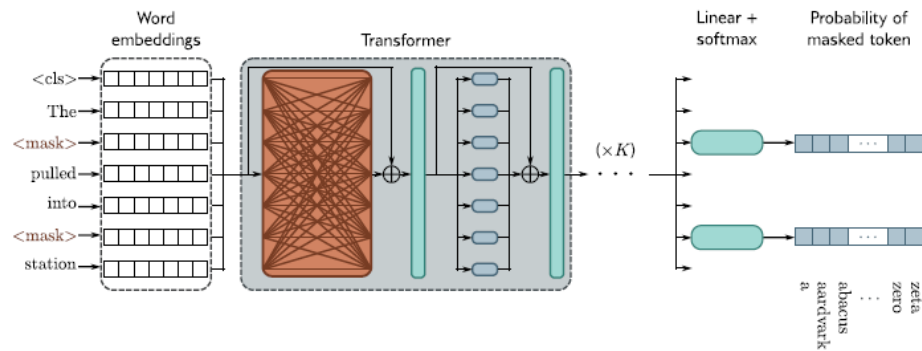
Encoders

- **Transformers** are neural networks that *compose* multiple transformer blocks to extract multiple levels of context from an input sequence
- The transformer blocks make up an **encoder**, which transforms text embeddings into latent state representations
- Latent states can then be passed through an output layer depending on desired task, e.g. softmax for classification
- Specific tasks may include sentiment analysis, named entity recognition, and text span prediction

BERT

- BERT is an encoder with a vocabulary of 30k tokens and 1024-dim embeddings
- 24 transformer layers with 16 heads each; 340M total parameters
- *Self-supervised learning* is used to train the model by randomly masking input tokens and tasking the model to predict them

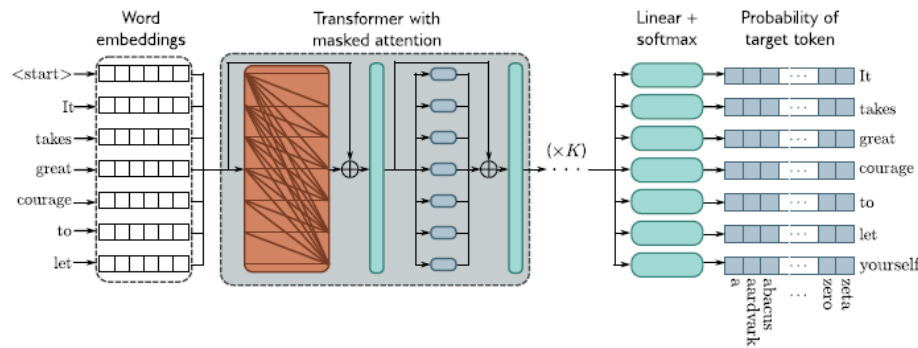
- This task forces the model to learn syntax and semantics about words
- Context comes from words both before and after a given mask



Decoders

- The goal of **decoders** is to *predict* the *next* token given an input sequence
- Unlike BERT, where context for a token comes from both left and right
- When training such a model, each token should only attend to *previous* ones
- We can enforce this constraint using *masked self-attention*

- Training objective is the likelihood that each predicted token matches the true one in the sequence



Large Language Models

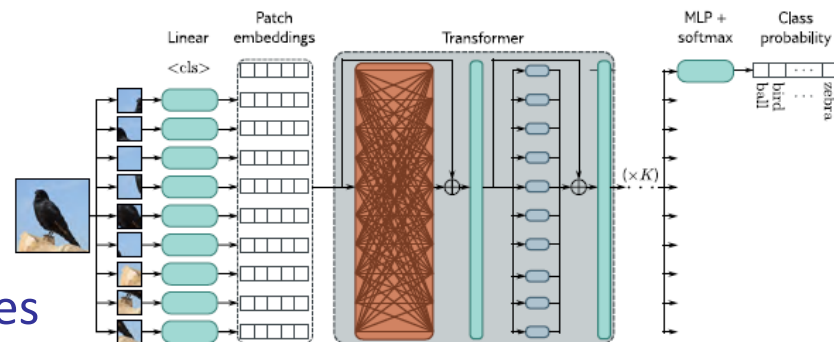
- Transformer and decoder models trained on large amounts of textual data can perform language tasks, foundation of modern **large language models**
- Given a prompt, the model predicts the next token by sampling from the resultant probability distribution
- Feed the prompt extended by the token back into the model and repeat
- Can be very efficient as prior embeddings can be reused each time
- GPT-3 (2020) accommodates sequence lengths of 2048 tokens and has 96 transformer layers with 96 heads each, for a total of 175B parameters

Vision Transformers

- **Vision transformers** process image data, but there are typically many more pixels in an image than words or tokens in a text sequence
- Solution: Divide an image into *patches* and compute embeddings for each
- Unlike convnets, the *inductive bias* of spatial invariance is not built in
- Vision transformers require much more data to learn these biases
- OTOH, unlike BERT or language models, vision transformers can be trained using “regular” supervised learning on labeled image data
- With enough training, performance can surpass convnets

Vision Transformers

- A convnet applies small filters to input and gradually increases receptive fields
- Builds understanding starting from local patterns and composing them together to see global structure
- Vision transformers apply self-attention across patches, so global interactions can be found from the start
- Less inductive bias, more flexible, and possibly more robust than convnets
- Many modern hybrids of both approaches



Summary

- Textual data can be split into tokens and converted into embeddings
- Tokens are projected into different query, key, and value representations
- Self-attention computes (softmax-normalized) weighted values, with weights based on pairwise similarities between queries and keys
- Multi-headed self-attention performs this across multiple subspaces
- Transformer layers combine self-attention with feedforward networks
- Transformers stack multiple transformer layers together; encoders produce contextual representations, while decoders generate sequences